❶ Registers the IHttpClientFactory service in DI

❷ Injects the IHttpClientFactory using DI

❸ Creates an HttpClient instance with an HttpClientHandler managed by the factory

❹ Configures the HttpClient for calling the API as before

❺ Uses the HttpClient in exactly the same way you would otherwise

The immediate benefit of using `IHttpClientFactory` in this way is efficient socket and DNS handling. When you create an `HttpClient` using `CreateClient()`, `IHttpClientFactory` uses a pooled `HttpClientHandler` to create a new instance of an `HttpClient`, pooling and disposing the handlers as necessary to find a balance between the tradeoffs described in section 33.1.

Minimal changes should be required to take advantage of this pattern, as the bulk of your code stays the same. Only the code where you're creating an `HttpClient` instance changes. This makes it a good option if you're refactoring an existing app.

`SocketsHttpHandler` vs. `IHttpClientFactory`

The limitations of `HttpClient` described in section 33.1 apply specifically to the `HttpClientHandler` at the end of the `HttpClient` handler pipeline in older versions of .NET Core. `IHttpClientFactory` provides a mechanism for managing the lifetime and reuse of `HttpClientHandler` instances.

From .NET 5 onward, the legacy `HttpClientHandler` has been replaced by `SocketsHttpHandler`. This handler has several advantages, most notably performance benefits and consistency across platforms. The `SocketsHttpHandler` can also be configured to use connection pooling and recycling, like `IHttpClientFactory`.